FitBuddy-AI Fitness Plan Generator Using Gemini Models

Government Arts and Science College-Manur-MSU142

TeamLeader:

Sandhiya P NM ID:F24D0CD882B6C7E9B9F44821FF74E70B

Team Members:Santhanamari G NM ID: C1A1D3256DDD23E87C0472B52EDBF20B

Saraswathi P NM ID: F7D8EF3617B74F0922A414AB1EDEF80E

Chinnathai S NM ID:6A593ADCC611910BA8240846346DA660

Muthumari M NM ID: 61F719397D001D3BE07494880A4743B7

Requirement Analysis Phase

Perfect — let's move into the *Requirement Analysis Phase* for your FitBuddy – AI Fitness Plan Generator. This phase ensures we clearly define what the system must achieve before development begins. Here's a structured breakdown:

📌 Functional Requirements

- *User Input & Personalization*
  - Collect user details: name, age, weight, fitness goal, workout intensity.
  - Generate a *7-day structured workout plan* (warm-up, main workout, cooldown).
  - Provide *nutrition/recovery tips* aligned with fitness goals.
  - Accept *feedback* (e.g., "add more cardio") and update plans dynamically.

- *Admin/Trainer Features*
  - Dashboard to view all users and their plans.
  - Compare original vs. updated workout plans.
  - Track user progress and activity.

- *Data Management*

- Store user profiles and workout plans in SQLite via SQLAlchemy ORM.
- Maintain both original and updated plans for reference.
- Ensure secure handling of sensitive data (API keys, user info).

⚙️ Non-Functional Requirements

- *Performance*
  - Fast response times for workout generation and updates.
  - Lightweight API calls using Gemini Flash for quick nutrition tips.

- *Scalability*
  - Modular architecture to support future cloud deployment.
  - Ability to integrate with wearables or external APIs.

- *Usability*
  - Clean, responsive UI (HTML + CSS + Jinja2).
  - Mobile-friendly design with intuitive navigation.

- *Security*
  - Environment variables for API key management.
  - Validation of user inputs using Pydantic schemas.

🎯 System Constraints

- Dependency on *Google Gemini APIs* for AI generation.
- Local deployment limited to FastAPI + Uvicorn unless extended to cloud.

- Requires Python environment with libraries: FastAPI, SQLAlchemy, Jinja2, Uvicorn, Google Generative AI SDK.

🔮 Future Enhancements (Identified Early)

- Cloud deployment for wider accessibility.
- Integration with wearable devices for real-time fitness tracking.
- Advanced analytics dashboards for trainers and users.
- Gamification features (badges, streaks, challenges).